

Lan honen helburua, gure proiektuko DataAccess klasean agertzen diren "Bad Smell" edo "issue" batzuk errefaktORIZAZIO batzuk aplikatuz ezabatzea da.

Taldeakideak: Qian Lizhang eta Iker Sada

Proiektuaren github helbidea: <https://github.com/IkerSada/JUnitTest.git>

Bad smell duen kodeak

"Write short units of code"

1- Hasierako kodea:

Metodo aukeratua: *alertakEguneratu(Ride r)*

```
public void alertakEguneratu(Ride r) {
    open();
    TypedQuery<Alerta> query = db.createQuery("SELECT a FROM Alerta a",
Alerta.class);
    db.getTransaction().begin();
    Integer bidaiZenbaki = r.getRideNumber();
    Ride r2 = db.find(Ride.class, bidaiZenbaki);
    String from = r2.getFrom();
    String to = r2.getTo();
    Date date = r2.getDate();
    List<Alerta> alertak = query.getResultList();
    for(Alerta a: alertak) {
        String nondik = a.getNondik();
        String nora = a.getNora();
        Date data = a.getDate();
        if(nondik.equals(from) && nora.equals(to) && egunBerdinakDira(data,date)) {
            a.setAbisatuta(true);
        }
    }
    db.getTransaction().commit();
    close();
}
```

2. ErrefaktORIZATUKO kodea:

```
public void alertakEguneratu(Ride r) {
    open();
    TypedQuery<Alerta> query = db.createQuery("SELECT a FROM Alerta a",
Alerta.class);
    db.getTransaction().begin();
    Integer bidaiZenbaki = r.getRideNumber();
    Ride r2 = db.find(Ride.class, bidaiZenbaki);
    String from = r2.getFrom();
```

```

        String to = r2.getTo();
        Date date = r2.getDate();
        List<Alerta> alertak = query.getResultList();
        alertakAbisatutaJarri(from, to, date, alertak);
        db.getTransaction().commit();
        close();
    }
    public void alertakAbisatutaJarri(String from, String to, Date date, List<Alerta>
alertak) {
        for(Alerta a: alertak) {
            String nondik = a.getNondik();
            String nora = a.getNora();
            Date data = a.getDate();
            if(nondik.equals(from) && nora.equals(to) &&
egunBerdinakDira(data,date)) {
                a.setAbisatuta(true);
            }
        }
    }
}

```

3. Egindako errefaktORIZAZIOREN deskribapena

Gure [alertakEguneratu\(\)](#) metodoa 15 kode lerro baino gehiago ditu eta hori sahiesteko, **“Extracting a Method from Code”** errefaktORIZAZIOA erabili dugu, metodoa laburtzeko, horrekin [alertakAbisatutaJarri\(\)](#) metodoa sortu digu [alertakEguneratu\(\)](#) metodoko zati bat hartuz.

4. Egilea: Qian Lizhang

“Write simple units of code”

Gure proiektuan ez ditugu aurkitu metodoren bat "Write simple units of code" motako “Bad smell” duena, gure metodoan if baten barruan gehienez 1 edo 2 if egoten dira, eta horrekin ez du adarkadura-puntu kopurua 4 duen egoerara iristen.

“Duplicate code”

1- Hasierako kodea:

Metodo aukeratua: [loadAlertak\(Bidaiaria b\)](#)

Metodo hau domain paketeko AlertakIkusiGUI klaseko metodoa da, zeren dataAccess klasean ez dugu aurkitzen “Write simple units of code” motako “bad smell” kodea.

```

private void loadAlertak(Bidaiaria b) {

```

```

        BLFacade facade = MainGUI.getBusinessLogic();
        List<Alerta> alertak = facade.alertakusi(b);
        String[] columnNames =
{ResourceBundle.getBundle("Etiquetas").getString("AlertakusiGUI.AlertNumber"),
        ResourceBundle.getBundle("Etiquetas").getString("From"),
        ResourceBundle.getBundle("Etiquetas").getString("To"),
        ResourceBundle.getBundle("Etiquetas").getString("Date"),

ResourceBundle.getBundle("Etiquetas").getString("AlertakusiGUI.Notified")};
        model = new DefaultTableModel(columnNames, 0);
        tableBalorazioak.setModel(model);
        model.setRowCount(0);
        if (alertak != null && !alertak.isEmpty()) {
            for (Alerta a : alertak) {
                Object[] row = {
                    a.getAlertazbk(),
                    a.getNondik(),
                    a.getNora(),
                    a.getDate(),
                    a.isAbisatuta()
                };
                model.addRow(row);
            }
            tableBalorazioak.setRowSelectionInterval(0, 0);
        }
    }
}

```

2. Errefaktoretzeko kodea:

```

    private void loadAlertak(Bidaiaria b) {
        BLFacade facade = MainGUI.getBusinessLogic();
        List<Alerta> alertak = facade.alertakusi(b);
        String etiketa = "Etiquetas";
        String[] columnNames =
{ResourceBundle.getBundle(etiketa).getString("AlertakusiGUI.AlertNumber"),
        ResourceBundle.getBundle(etiketa).getString("From"),
        ResourceBundle.getBundle(etiketa).getString("To"),
        ResourceBundle.getBundle(etiketa).getString("Date"),

ResourceBundle.getBundle(etiketa).getString("AlertakusiGUI.Notified")};
        model = new DefaultTableModel(columnNames, 0);
        tableBalorazioak.setModel(model);
        model.setRowCount(0);
        if (alertak != null && !alertak.isEmpty()) {
            for (Alerta a : alertak) {
                Object[] row = {

```

```

        a.getAlertazbk(),
        a.getNondik(),
        a.getNora(),
        a.getDate(),
        a.isAbisatuta()
    };
    model.addRow(row);
}
tableBalorazioak.setRowSelectionInterval(0, 0);
}
}

```

3. Egindako errefaktORIZAZIOREN deskribapena

loadAlertak() metodo honek lerro askotan "Etiquetas" errepikatzen ditu eta hori konpontzeko "Extract Local Variable" errefaktORIZAZIOA erabili dugu, etiketa izeneko eta bere balioa "Etiquetas" den aldagaia sortuz.

4. Egilea: Qian Lizhang

"Keep unit interfaces small"

1- Hasierako kodea:

Metodo aukeratua: createRide(String from, String to, Date date, int nPlaces, float price, String driverEmail)

```

    public boolean erregistratu(String izena, String abizena, Date jaiotzeData, String sexua,
String erabiltzaileMota, String email, String pasahitza) {
    open();
    if(erabiltzaileMota.equals("Gidaria")) {
        Driver u = db.find(Driver.class,email);
        if (u==null) {
            db.getTransaction().begin();
            Driver d=new Driver(izena, abizena, jaiotzeData, sexua, email,
pasahitza);

            db.persist(d);
            db.getTransaction().commit();
            close();
            return true;
        } else {
            return false;
        }
    } else {
        Bidaiaaria u = db.find(Bidaiaaria.class,email);
        if (u==null) {
            db.getTransaction().begin();
            Bidaiaaria b=new Bidaiaaria(izena, abizena, jaiotzeData, sexua, email,
pasahitza);

            db.persist(b);

```

```

        db.getTransaction().commit();
        close();
        return true;
    } else {
        return false;
    }
}
}

```

2. Errefaktoretzeko kodea:

Sortutako klase berria (ErabiltzaileData) eta bere implementazioa:

```

package domain;
import java.util.Date;
public class ErabiltzaileData {
    public String izena;
    public String abizena;
    public Date jaiotzeData;
    public String sexua;
    public String erabiltzaileMota;
    public String email;
    public String pasahitza;
    public ErabiltzaileData(String izena, String abizena, Date jaiotzeData,
        String sexua, String erabiltzaileMota,
        String email, String pasahitza) {
        this.izena = izena;
        this.abizena = abizena;
        this.jaiotzeData = jaiotzeData;
        this.sexua = sexua;
        this.erabiltzaileMota = erabiltzaileMota;
        this.email = email;
        this.pasahitza = pasahitza;
    }
}

```

DataAccess-ean egin ditugun aldaketak:

```

    public boolean erregistratu(ErabiltzaileData data) {
        open();
        if(data.erabiltzaileMota.equals("Gidaria")) {
            Driver u = db.find(Driver.class, data.email);
            if (u==null) {
                db.getTransaction().begin();
                Driver d=new Driver(data.izena, data.abizena,
data.jaiotzeData, data.sexua, data.email, data.pasahitza);
                db.persist(d);
            }
        }
    }
}

```

```

        db.getTransaction().commit();
        close();
        return true;
    } else {
        return false;
    }
} else {
    Bidaiaria u = db.find(Bidaiaria.class, data.email);
    if (u == null) {
        db.getTransaction().begin();
        Bidaiaria b = new Bidaiaria(data.izena, data.abizena,
data.jaiotzeData, data.sexua, data.email, data.pasahitza);
        db.persist(b);
        db.getTransaction().commit();
        close();
        return true;
    } else {
        return false;
    }
}
}
}

```

3. Egindako errefaktORIZAZIOREN deskribapena

Aukeratu metodo honek 7 parametro ditu, “bad smell” konpontzeko metodo egoki bat da. Hau konpontzeko ErabiltzaileData klase berri bat sortu eta bere atributuak eta eraikitzaileak definitzen dugu. Hori egin ezkerro, erregistratu() metodoan 7 parametroa izan beharrean, parametro bakarrera murrizten dugu, ErabiltzaileData motakoa, eta prozesu honek “Change Method Signature” errefaktorea erabiliz.

4. Egilea: Qian Lizhang

“Write short units of code”

1- Hasierako kodea:

Metodo aukeratua: *erreserbaEgin(int bidaiZenbaki, Bidaiaria bidaiari2, int eserlekuKop)*

```
public void erreserbaEgin(int bidaiZenbaki, Bidaiaria bidaiari2, int eserlekuKop) {  
    open();  
    String email = bidaiari2.getEmail();  
    Bidaiaria bidaiari = db.find(Bidaiaria.class, email);  
    Ride r = db.find(Ride.class, bidaiZenbaki);  
    Erreserba e = bidaiari.erreserbaBilatu(bidaiZenbaki);  
    db.getTransaction().begin();  
    if(e==null) {  
        e = bidaiari.addErreserba(r, eserlekuKop);  
    } else {  
        if(e.getEgoera().equals("deuseztatu")) {  
            e.setnPlaces(0);  
        }  
        e.setEgoera("itxaron");  
        e.updatePlaces(eserlekuKop);  
    }  
    r.updateSeat(eserlekuKop);  
    float prezioa = r.getBidaiarenPrezioa(eserlekuKop);  
    bidaiari.diruaKendu(prezioa);  
    e.eguneratuDiruloztuta(prezioa);  
    bidaiari.addMugimendua("Diruzorrotik erreserbaren prezioa kobratu da (diru  
hori izoztuta)");  
    db.getTransaction().commit();  
    close();  
}
```

2. Errefaktoretzatuko kodea:

```
public void erreserbaEgin(int bidaiZenbaki, Bidaiaria bidaiari2, int eserlekuKop) {  
    open();  
    db.getTransaction().begin();  
    String email = bidaiari2.getEmail();  
    Bidaiaria bidaiari = getBidaiaria(email);  
    Ride r = db.find(Ride.class, bidaiZenbaki);  
    Erreserba e = eguneratuErreserba(bidaiZenbaki, eserlekuKop, bidaiari, r);  
    eguneratuDiruaEtaMugimendua(eserlekuKop, bidaiari, r, e);  
    db.getTransaction().commit();  
    close();  
}  
  
private Erreserba eguneratuErreserba(int bidaiZenbaki, int eserlekuKop, Bidaiaria bidaiari,  
Ride r) {  
    Erreserba e = bidaiari.erreserbaBilatu(bidaiZenbaki);  
    if(e==null) {
```

```

        e = bidaiari.addErreserba(r, eserlekuKop);
    } else {
        if(e.getEgoera().equals("deuseztatu")) {
            e.setnPlaces(0);
        }
        e.setEgoera("itxaron");
        e.updatePlaces(eserlekuKop);
    }
    r.updateSeat(eserlekuKop);
    return e;
}
private void eguneratuDiruaEtaMugimendua(int eserlekuKop, Bidaiaria bidaiari, Ride r,
Erreserba e) {
    float prezioa = r.getBidairenPrezioa(eserlekuKop);
    bidaiari.diruaKendu(prezioa);
    e.eguneratuDirulzoztua(prezioa);
    bidaiari.addMugimendua("Diruzorrotik erreserbaren prezioa kobratu da (diru hori
izoztuta)");
}
private Bidaiaria getBidaiaria(String email) {
    Bidaiaria bidaiari = db.find(Bidaiaria.class, email);
    return bidaiari;
}

```

3. Egindako errefaktORIZAZIOREN deskribapena

Gure erreserbaEgin() metodoa 25 lerro baino gehiago zituen eta hainbat erantzukizun zituen, eta hori saihesteko “**Extracting a Method from Code**” errefaktORIZAZIOA erabili dugu. Metodo nagusia laburtzeko, honako sub-metodoak sortu ditugu: **getBidaiaria()**, **eguneratuErreserba()** eta **eguneratuDiruaEtaMugimendua()**, erreserbaEgin() metodoko logikoki lotutako zatiak hartuz.

Horrela, erreserbaEgin() metodo nagusia argiagoa eta laburragoa bihurtu da, kodearen irakurgarritasuna eta mantengarritasuna hobetu dira, eta zati bakoitza bere funtzio espezifikoan dago.

4. Egilea: Iker Sada

"Write simple units of code"

Gure proiektuan ez ditugu aurkitu metodoren bat "Write simple units of code" motako "Bad smell" duena, gure metodoan if baten barruan gehienez 1 edo 2 if egoten dira, eta horrekin ez du adarkadura-puntu kopurua 4 duen egoerara iristen.

"Duplicate code"

1- Hasierako kodea:

```
private void loadErreserbak(Bidaia b) {
    BLFacade facade = MainGUI.getBusinessLogic();
    List<Erreserba> erreserbak = facade.getBidaiaErreserbak(b);
    String[] columnNames =
{ResourceBundle.getBundle("Etiquetas").getString("From"),
    ResourceBundle.getBundle("Etiquetas").getString("To"),
    ResourceBundle.getBundle("Etiquetas").getString("Date"),
    ResourceBundle.getBundle("Etiquetas").getString("Price"),
    ResourceBundle.getBundle("Etiquetas").getString("Traveler"),
    ResourceBundle.getBundle("Etiquetas").getString("NumberOfPlaces"),
    ResourceBundle.getBundle("Etiquetas").getString("BookNumber"),
    ResourceBundle.getBundle("Etiquetas").getString("Status")};
    model = new DefaultTableModel(columnNames, 0);
    tableErreserbaOnartuak.setModel(model);
    model.setRowCount(0);
    if (erreserbak != null && !erreserbak.isEmpty()) {
        for (Erreserba e : erreserbak) {
            if (e.getBidaia() != null) {
                Object[] row = {
                    e.getBidaia().getFrom(),
                    e.getBidaia().getTo(),
                    e.getBidaia().getDate(),
                    e.getBidaia().getPrice(),
                    e.getTraveler().getIzena(),
                    e.getnPlaces(),
                    e.getBookNumber(),
                    e.getEgoera()
                };
                model.addRow(row);
            }
        }
        if (model.getRowCount() > 0) {
            tableErreserbaOnartuak.setRowSelectionInterval(0, 0);
        }
    }
}
```

2. Errefaktoretzatuko kodea:

```
private void loadErreserbak(Bidaia b) {
    String etiketa = "Etiquetas";
```

```

        BLFacade facade = MainGUI.getBusinessLogic();
        List<Erreserba> erreserbak = facade.getBidaiaIarenErreserbak(b);
        String[] columnNames =
{ResourceBundle.getBundle("Etiquetas").getString("From"),
        ResourceBundle.getBundle(etiketa).getString("To"),
        ResourceBundle.getBundle(etiketa).getString("Date"),
        ResourceBundle.getBundle(etiketa).getString("Price"),
        ResourceBundle.getBundle(etiketa).getString("Traveler"),

ResourceBundle.getBundle(etiketa).getString("NumberOfPlaces"),
        ResourceBundle.getBundle(etiketa).getString("BookNumber"),
        ResourceBundle.getBundle(etiketa).getString("Status")};

        model = new DefaultTableModel(columnNames, 0);
        tableErreserbaOnartuak.setModel(model);
        model.setRowCount(0);
        if (erreserbak != null && !erreserbak.isEmpty()) {
            for (Erreserba e : erreserbak) {
                if (e.getBidaia() != null) {
                    Object[] row = {
                        e.getBidaia().getFrom(),
                        e.getBidaia().getTo(),
                        e.getBidaia().getDate(),
                        e.getBidaia().getPrice(),
                        e.getTraveler().getIzena(),
                        e.getnPlaces(),
                        e.getBookNumber(),
                        e.getEgoera()
                    };
                    model.addRow(row);
                }
            }
            if (model.getRowCount() > 0) {
                tableErreserbaOnartuak.setRowSelectionInterval(0, 0);
            }
        }
    }
}

```

3. Egindako errefaktORIZAZIOREN deskribapena

loadErreserbak() metodo honek lerro askotan "Etiquetas" errepikatzen ditu eta hori konpontzeko "**Extract Local Variable**" errefaktORIZAZIOA erabili dugu, etiketa izenekoa eta bere balioa "Etiquetas" den aldagaia sortuz.

4. Egilea: Iker Sada

"Keep unit interfaces small"

1- Hasierako kodea:

Metodo aukeratua: erregistratu(String izena, String abizena, Date jaiotzeData, String sexua, String erabiltzaileMota, String email, String pasahitza)

```
public boolean erregistratu(String izena, String abizena, Date jaiotzeData, String sexua, String
erabiltzaileMota, String email, String pasahitza) {
    open();
    if(erabiltzaileMota.equals("Gidaria")) {
        Driver u = db.find(Driver.class,email);
        if (u==null) {
            db.getTransaction().begin();
            Driver d=new Driver(izena, abizena, jaiotzeData, sexua, email, pasahitza);
            db.persist(d);
            db.getTransaction().commit();
            close();
            return true;
        } else {
            return false;
        }
    } else {
        Bidaiaaria u = db.find(Bidaiaaria.class,email);
        if (u==null) {
            db.getTransaction().begin();
            Bidaiaaria b=new Bidaiaaria(izena, abizena, jaiotzeData, sexua, email, pasahitza);
            db.persist(b);
            db.getTransaction().commit();
            close();
            return true;
        } else {
            return false;
        }
    }
}
```

2. Errefaktoretzeko kodea:

Sortutako klase berria (UserData) eta bere implementazioa:

```
package dataAccess;

import java.util.Date;

public class UserData {
    private String izena;
    private String abizena;
    private Date jaiotzeData;
    private String sexua;
    private String erabiltzaileMota;
    private String email;
    private String pasahitza;

    public UserData(String izena, String abizena, Date jaiotzeData, String sexua,
        String erabiltzaileMota, String email, String pasahitza) {
        this.izena = izena;
        this.abizena = abizena;
        this.jaiotzeData = jaiotzeData;
        this.sexua = sexua;
        this.erabiltzaileMota = erabiltzaileMota;
        this.email = email;
        this.pasahitza = pasahitza;
    }
    // Getters
    public String getIzena() { return izena; }
    public String getAbizena() { return abizena; }
    public Date getJaiotzeData() { return jaiotzeData; }
    public String getSexua() { return sexua; }
    public String getErabiltzaileMota() { return erabiltzaileMota; }
    public String getEmail() { return email; }
    public String getPasahitza() { return pasahitza; }
}
```

DataAccess-ean egin ditugun aldaketak:

```
public boolean erregistratu(UserData data) {
    open();
    String email = data.getEmail();

    if (data.getErabiltzaileMota().equals("Gidaria")) {
        Driver u = db.find(Driver.class, email);
        if (u == null) {
            db.getTransaction().begin();
            Driver d = new Driver(
                data.getIzena(),
                data.getAbizena(),
                data.getJaiotzeData(),
                data.getSexua(),
                data.getEmail(),
                data.getPasahitza()
            );
            db.persist(d);
            db.getTransaction().commit();
            close();
            return true;
        } else {
            close();
            return false;
        }
    } else { // Bidaiaaria
        Bidaiaaria u = db.find(Bidaiaaria.class, email);
        if (u == null) {
            db.getTransaction().begin();
            Bidaiaaria b = new Bidaiaaria(
                data.getIzena(),
                data.getAbizena(),
                data.getJaiotzeData(),
                data.getSexua(),
                data.getEmail(),
                data.getPasahitza()
            );
            db.persist(b);
            db.getTransaction().commit();
            close();
            return true;
        } else {
            close();
            return false;
        }
    }
}
```

3. Egindako errefaktORIZAZIOREN deskribapena

Lehen, erregistratu() metodoak 7 parametro jasotzen zituen, eta hori “bad smell” bat zen kodearen mantentze-lanak zailtzen zuena. Arazoa konpontzeko, ErabiltzaileData klase berria sortu dugu, bere atributuak eta eraikitzailea definituz. Orain, erregistratu() metodoak parametro bakarra jasotzen du, ErabiltzaileData motakoa, eta horrela “Change Method Signature” errefaktORIZAZIOA erabili dugu. Honek kodearen irakurgarritasuna, mantentze-lanak eta berrerabilgarritasuna hobetzen ditu.

4. Egilea: Iker Sada